

[1 일차 오후] Amazon Comprehend 를 활용한 텍스트 AI 실습

문서·이메일·리뷰·상담기록 같은 **비정형 텍스트**를 코드 몇 줄로 구조화된 정보로 바꾸는 **AWS 관리형 NLP 서비스 Amazon Comprehend** 를 5 개의 **기업 미니프로젝트**로 실습합니다.

개요

기본 정보

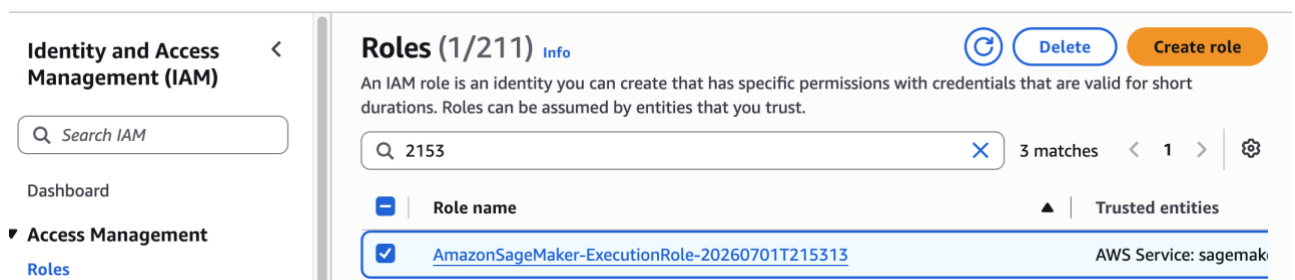
항목	내용
세션명	Amazon Comprehend 를 활용한 텍스트 AI (감성·개체·PII·종합 파이프라인)
소요시간	약 3 시간
실습 도구	SageMaker Studio 의 JupyterLab (Python 3, boto3)
실습 데이터	글로벌 상품 리뷰 · 보도자료 · 영문 상담기록 · 고객 멘션 텍스트

학습 목표

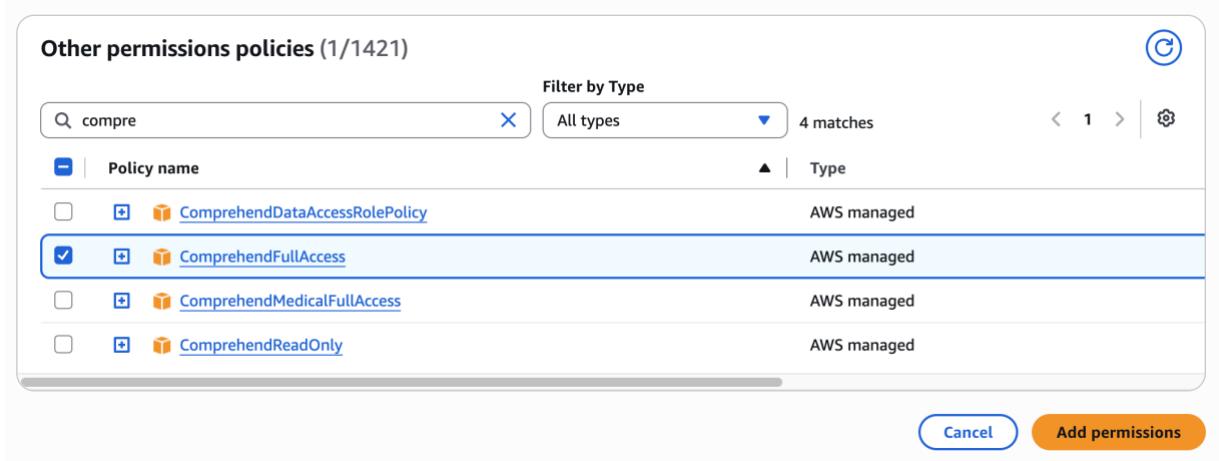
- 관리형 NLP 서비스 Comprehend 의 개념과 **기업에서의 활용 방식**을 이해한다.
- 감성 분석 · 언어 감지 · 개체 인식(NER) · 핵심 문구 · 대상 감성 · PII 감지 · 구문 분석 API 를 직접 호출한다.
- 여러 API 를 묶어 **VOC 자동 분석 파이프라인**을 구현하고 결과를 시각화한다.
- 지도학습(분류)과 비지도학습(토픽 모델링·군집)의 차이를 이해하고 라벨 없이 주제를 자동 발견한다.
- 각 기능이 실제 기업 시스템(이커머스·미디어·금융)에서 어떻게 쓰이는지 연결해 설명할 수 있다.

사전 준비사항

- AWS 계정 로그인 (강사 제공) — **1 일차 오전에 생성한 SageMaker 도메인 그대로 사용**
- 실행 역할(IAM Role)에 **ComprehendFullAccess** 권한 부여 (오전 환경설정에서 완료, 그렇지 않다면)
 - IAM > Roles > 검색



- Role 을 선택하고, [Add permissions > Attach policies]



이 강의의 핵심: 5 개 실습 = 5 개 기업 미니프로젝트

같은 텍스트 AI 라도 “회사에서 실제로 이렇게 쓴다”를 체감하도록, 각 Lab 을 하나의 업무 시나리오로 구성했습니다.

Lab	기업 시나리오	핵심 API	비고
Lab 1	이커머스 — 글로벌 리뷰 자동 분석 (언어 라우팅·부정 우선대응·배치)	detect_dominant_language / sentiment / batch	
Lab 2	미디어·PR — 브랜드 모니터링 (기업·인물·제품 추출, 이슈 요약)	detect_entities / key_phrases	
Lab 3	금융·공공 — 상담 컴플라이언스 (항목별 만족도·PII 자동 마스킹)	targeted_sentiment / pii_entities	
Lab 4	데이터팀 — VOC 통합 분석 파이프라인 (4 개 API 통합·대시보드)	detect_syntax + 통합	
Lab 5	CX — 라벨 없이 리뷰 주제 자동 발견 (비지도 군집 체험)	start_topics_detection_job + KMeans	

1. 실습 환경 준비

오전에 만든 SageMaker Studio 를 열고, 실습 노트북을 업로드한 뒤 첫 셀을 실행해 환경을 확인합니다.
새 계정·도메인을 만들 필요가 없습니다.

1.1 SageMaker Studio 열기

1. AWS 콘솔 상단 검색창에 "SageMaker" 입력 → Amazon SageMaker AI 클릭
2. 좌측 메뉴에서 Admin configurations → Domains → 오전에 생성한 본인 도메인 클릭
3. User profiles 탭에서 본인 프로필의 Launch → Studio 클릭

1.2 JupyterLab 스페이스 실행

4. Studio 좌측 상단 JupyterLab 클릭
5. Create JupyterLab space → 이름 입력(예: comprehend-lab) → Create space
6. 인스턴스가 Running 되면 Open JupyterLab 클릭 (1~2 분 소요)

1.3 실습 노트북 업로드

7. JupyterLab 좌측 파일 탐색기 상단의 업로드(↑) 아이콘 클릭
8. comprehend-lab1~4-student.ipynb 4 개 파일을 모두 업로드
9. lab1 노트북을 더블클릭해서 열기 → 커널이 Python 3 인지 확인

1.4 첫 실행 — 환경 초기화 & 한글 폰트

첫 번째 코드 셀(✅ 환경 초기화)을 선택하고 **Shift + Enter** 로 실행합니다.

```
# ✅ [제공 코드] 환경 초기화
import boto3, json
import matplotlib.pyplot as plt
import pandas as pd

# 한글 폰트 설정 (SageMaker Studio) — 최초 1 회만 설치, 이후 즉시 로드
try:
    import koreanize_matplotlib
except ImportError:
    import sys, subprocess
    subprocess.run([sys.executable, '-m', 'pip', 'install', '-q', 'koreanize-matplotlib'])
    import koreanize_matplotlib

comprehend = boto3.client('comprehend', region_name='ap-northeast-2')
print('✅ Comprehend 클라이언트 생성 완료 (서울 ap-northeast-2)')
```

💡 처음 실행이 조금 느린 이유

한글 그래프 폰트(koreanize-matplotlib)를 자동 설치하기 때문입니다. 최초 1 회만 설치되고, 이후에는 바로 로드됩니다.

✅ 표시와 “클라이언트 생성 완료” 가 보이면 성공입니다.

 자격증명을 따로 입력하지 않습니다

SageMaker Studio 는 **실행 역할(IAM Role)**로 AWS 에 인증되므로 access key 를 코드에 넣지 않습니다.
로컬 PC 와 가장 다른 점입니다.

2. 실습 1 — 글로벌 이커머스 리뷰 자동 분석

소요시간 약 30 분 | 난이도 ★★ | 리전 서울 ap-northeast-2

시나리오 — 글로벌 쇼핑몰 'ShopGlobal' CX 팀

당신은 글로벌 온라인 쇼핑몰의 고객경험(CX) 데이터 분석가입니다. 매일 수만 건의 리뷰가 여러 언어로 쏟아집니다.

① 언어 감지로 담당 언어 CS 팀에 라우팅 → ② 감성으로 부정 리뷰 우선 대응 → ③ 배치로 대량 처리.

배우는 API

- `detect_dominant_language` — 텍스트의 언어 자동 감지
- `detect_sentiment` — 긍정/부정/중립/혼합 감성 분석
- `batch_detect_sentiment` — 여러 문서(최대 25 개)를 한 번에 분석

TODO 1 — 언어 감지: 어느 나라 리뷰인지 자동 분류

빈칸 ____ 을 채워 다국어 리뷰의 언어를 감지하세요. 채울 값은 코드 옆 # ← 힌트 주석에 있습니다.

```
for text in texts:
    response = comprehend.detect_dominant_language(
        Text=____ # ← text
    )
    top = response['Languages'][0]
    lang_code = top[____] # ← 'LanguageCode'
    score = top[____] # ← 'Score'
```

실행하면 한국어 [ko], 영어 [en], 일본어 [ja] 처럼 언어 코드와 신뢰도가 출력됩니다.

TODO 2 — 감성 분석: 부정 리뷰 가려내기

```
response = comprehend.detect_sentiment(
    Text=____, # ← review
    LanguageCode=____ # ← 'ko'
)
sentiment = response[____] # ← 'Sentiment'
scores = response[____] # ← 'SentimentScore'
```

결과의 Sentiment 가 **NEGATIVE** 인 리뷰를 CS 우선 대응 큐로 보낸다고 생각해 보세요. 제공된 시각화 셀을 실행하면 4 개 리뷰의 감성 점수가 막대그래프로 나옵니다.

TODO 3 — 배치 처리: 수천 건을 한 번에

```
response = comprehend.batch_detect_sentiment(
    TextList=____, # ← batch_texts
    LanguageCode=____ # ← 'ko'
)
for item in response[____]: # ← 'ResultList'
```

 실무 연결

이 배치 분석을 매일 밤 **S3 → Lambda → Comprehend → DB → 대시보드** 로 자동 실행하면, 아침마다 '어제의 부정 리뷰 Top 10' 리포트가 자동 생성됩니다.

 정답 (먼저 스스로 풀어본 뒤 확인)

TODO1: Text=text · top['LanguageCode'] · top['Score']

TODO2: Text=review, LanguageCode='ko' · response['Sentiment'] · response['SentimentScore']

TODO3: TextList=batch_texts, LanguageCode='ko' · response['ResultList']

3. 실습 2 — 미디어·PR 브랜드 모니터링

소요시간 약 30 분 | 난이도 ★★☆☆ | 리전 서울 ap-northeast-2

시나리오 — 홍보(PR)팀 미디어 모니터링

당신은 홍보팀 담당자입니다. 매일 자사·경쟁사 기사와 SNS 가 수백 건 올라옵니다.

기사에서 기업·인물·제품을 자동 추출(NER)하고 핵심 이슈를 요약(key phrases)해 일일 미디어 브리핑을 자동화합니다.

배우는 API

- **detect_entities** — 인물·조직·장소·날짜·금액·제품명 등 개체 추출
- **detect_key_phrases** — 핵심 명사구(주제) 추출

TODO 1 — 개체 인식(NER)

```
response = comprehend.detect_entities(
    Text=____,      # ← news_text
    LanguageCode=____ # ← 'ko'
)
```

```
entities = response[____] # ← 'Entities'
```

for e in entities:

```
    print(f"[{e[____]}] {e['Text']} (신뢰도 {e['Score']:.3f})" # ← 'Type'
```

보도 문장에서 ORGANIZATION(삼성전자·애플), PERSON(이재용), DATE, QUANTITY 등이 유형별로 추출됩니다. 제공된 셀로 유형별 분포 막대그래프를 확인하세요.

TODO 2 — 핵심 문구 추출

```
response = comprehend.detect_key_phrases(
    Text=____,      # ← article
    LanguageCode=____ # ← 'ko'
)
```

```
phrases = response[____] # ← 'KeyPhrases'
```

실무 연결

ORGANIZATION 개체를 매일 집계 → ‘자사 vs 경쟁사 언급량 추이’ 그래프. 핵심 문구를 모으면 이슈 키워드 클라우드가 되어 부정 이슈를 조기에 감지합니다.

정답

TODO1: Text=news_text, LanguageCode='ko' · response['Entities'] · e['Type']

TODO2: Text=article, LanguageCode='ko' · response['KeyPhrases']

4. 실습 3 — 금융·공공 상담 컴플라이언스

소요시간 약 40 분 | 난이도 ★★ | 리전 버지니아 us-east-1

시나리오 — 금융 콜센터 품질·정보보호팀

당신은 콜센터 상담 품질·개인정보보호 담당자입니다. 매일 수천 건의 상담 기록이 쌓입니다.

① **항목별 감성**으로 고객이 무엇에 불만인지 파악 → ② 상담기록의 **개인정보(PII)**를 자동 탐지·마스킹해 규정을 준수합니다.

리전·언어 주의

detect_targeted_sentiment 와 detect_pii_entities 는 **영어(en)**만 지원합니다. 그래서 이 실습의 데이터는 영어이고, 리전은 **us-east-1** 입니다. (노트북 환경 셀에 이미 반영)

TODO 1 — 대상 감성(Targeted Sentiment): 항목별 만족도

```
response = comprehend.detect_targeted_sentiment(
    Text=____,      # ← review
    LanguageCode=____ # ← 'en'
)
entities = response[____] # ← 'Entities'
for e in entities:
    m = e['Mentions'][0]
    print(m[____], m['MentionSentiment'][____]) # ← 'Text', 'Sentiment'
```

camera → POSITIVE, battery → NEGATIVE 처럼 **개체(기능)별로 감성이 분리** 됩니다. 전체 문장 감성보다 훨씬 실무적입니다.

TODO 2 — PII 감지

```
response = comprehend.detect_pii_entities(
    Text=____,      # ← pii_text
    LanguageCode=____ # ← 'en'
)
for p_ent in response[____]: # ← 'Entities'
    start, end = p_ent[____], p_ent[____] # ← 'BeginOffset', 'EndOffset'
```

이름·이메일·전화번호·주소·카드번호가 **유형과 위치(offset)** 까지 탐지됩니다. TODO 3에서는 이 위치를 * 로 가리는 마스킹 함수를 구현합니다.

실무 연결

상담 로그를 분석팀에 넘기기 **전에** PII 를 자동 마스킹하면, 사람이 직접 가릴 필요가 없어 휴먼 에러와 개인정보보호법 위반 위험을 제거합니다.

정답

TODO1: Text=review, LanguageCode='en' · response['Entities'] · m['Text'], m['Sentiment']

TODO2: Text=pii_text, LanguageCode='en' · response['Entities'] · 'BeginOffset','EndOffset'

5. 실습 4 — VOC 통합 분석 파이프라인

소요시간 약 50 분 | 난이도 ★★★★★ | 리전 버지니아 us-east-1

시나리오 — 데이터팀의 VOC 자동 분석 시스템

당신은 데이터 엔지니어입니다. 지금까지 배운 기능을 하나의 함수로 묶어 고객의 소리(VOC)를 자동 분석하는 파이프라인을 만듭니다.

구문 분석 → **analyze_text()** 통합 함수 → JSON/CSV 적재 → 2×2 대시보드.

배우는 내용

- **detect_syntax** — 품사(POS) 태깅
- **analyze_text()** — 언어감지+감성+개체+핵심문구를 하나로 통합
- 결과를 results.json / results.csv 로 저장하고 대시보드(dashboard.png)로 시각화

TODO 2 — 통합 파이프라인 함수 (핵심)

```
def analyze_text(text, lang='ko'):
    result = {'text': text}
    # 1) 언어 감지
    detected = comprehend.detect_dominant_language(Text=_____)[0]['LanguageCode'] # ← text
    # 2) 감성
    s = comprehend.detect_sentiment(Text=_____, LanguageCode=_____) # ← text, detected
    result['sentiment'] = s['Sentiment']
    # 3) 개체 4) 핵심문구 ... (동일 패턴)
    return result
```

하나의 함수가 **언어→감성→개체→핵심문구** 를 순서대로 호출합니다. TODO 3 은 여러 텍스트를 반복 처리해 JSON/CSV 로 저장하고, TODO 4 는 결과를 **2×2 대시보드** 로 시각화합니다.

실무 연결 — 오늘 만든 것이 곧 실무 시스템

고객 리뷰·문의(S3) → Lambda → Comprehend → DB 적재 → BI 대시보드

마케팅팀은 감성 추이로 캠페인 반응을, CS 팀은 부정 급증 알람을, 보안팀은 PII 마스킹을 동시에 활용합니다.

축하합니다 — 여러분은 기업이 실제로 쓰는 텍스트 AI 파이프라인의 축소판을 완성했습니다. 🎉

6. 실습 5 — 토픽 모델링 & 비지도 학습(군집) 체험

소요시간 약 30 분 | 난이도 ★★☆☆ | 리전 서울 ap-northeast-2 / 로컬

시나리오 — 리뷰 속 '숨은 주제' 자동 발견

수천 건 리뷰에 누구도 '배송·품질·가격·상담' 라벨을 달아두지 않았습니다. 라벨 없이 비슷한 리뷰끼리 자동으로 묶어 주제를 파악합니다.

이것이 **비지도 학습 = 군집(클러스터링)** 이고, Comprehend 토픽 모델링이 관리형으로 해줍니다.

지도학습 vs 비지도학습

구분	지도학습 (Lab 1~4)	비지도학습 (Lab 5)
정답 라벨	있음 (긍정/부정 등)	없음
하는 일	분류·예측	비슷한 것끼리 묶기(군집)
예시	감성 분석·개체 인식	토픽 모델링·클러스터링

개요 — Comprehend 관리형 토픽 모델링

대량 문서는 `start_topics_detection_job` 으로 S3 에 올려 **비동기 잡**을 돌립니다. 완료에 보통 수십 분 이 걸리므로, 실습에서는 같은 개념(비지도 군집)을 즉시 체험합니다.

TODO 1 — 문서 벡터화(TF-IDF): 텍스트 → 숫자

```
vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b", stop_words=STOPWORDS)
X = vectorizer.fit_transform(____) # ← sample_docs
```

문장을 숫자 벡터로 바꿔야 군집을 할 수 있습니다. TF-IDF 는 **문서에서 특징적인 단어**에 가중치를 줍니다. (불용어는 미리 제거)

TODO 2 — KMeans 군집화: 라벨 없이 토픽 묶기

```
km = KMeans(n_clusters=____, random_state=42, n_init=10) # ← NUM_TOPICS(=4)
labels = km.fit_predict(____) # ← X
```

KMeans 가 비슷한 리뷰끼리 4 개 그룹으로 묶습니다. 제공된 셀로 **토픽별 대표 단어** 와 군집 크기를 확인하면 배송·품질·가격·상담 주제가 드러납니다.

TODO 3 — 새 리뷰의 토픽 예측

```
new_X = vectorizer.transform(____) # ← new_reviews
pred = km.predict(____) # ← new_X
```

실무 연결

수천 건 리뷰 → 토픽 모델링(군집) → 토픽별 비중·추이 → 대시보드. 라벨을 못 만드는 **새 이슈도 군집으로 드러납니다**. 군집 + 감성(Lab 1)을 결합하면 '어떤 주제가 부정적인지'까지 파악.

 정답

```

TODO1: fit_transform(sample_docs)
TODO2: n_clusters=NUM_TOPICS · fit_predict(X)
TODO3: transform(new_reviews) · predict(new_X)

```

7. 마무리 & 트러블슈팅

자주 묻는 오류

증상	원인 / 해결
AccessDeniedException	실행 역할에 ComprehendFullAccess 권한 누락 → IAM 역할에 정책 추가
UnsupportedLanguageException	targeted_sentiment-P11 에 한국어 입력 → 영어(en) + us-east-1 사용
그래프 한글이 □로 깨짐	환경 초기화 셀(koreanize 설치)을 먼저 실행했는지 확인
ResourceNotFound / 리전 오류	Lab 3·4 는 us-east-1, Lab 1·2 는 ap-northeast-2 — 노트북별 리전 확인
배치 호출 오류	batch_detect_sentiment 는 한 번에 최대 25 개 문서까지

심화 도전

- 실제 상품 리뷰 10 개를 입력해 긍정/부정 비율을 파이차트로 시각화하기
- 가장 많이 언급된 ORGANIZATION Top 5 추출하기
- analyze_text() 파이프라인을 **AWS Lambda** 함수로 배포하는 아키텍처 설계하기